

Production EKS Platform

Structured Learning Notes and Interview Guide

AWS Networking • EKS • Kubernetes • ECR • ALB • Autoscaling • Monitoring

Cleaned and organized for practical learning and interview revision

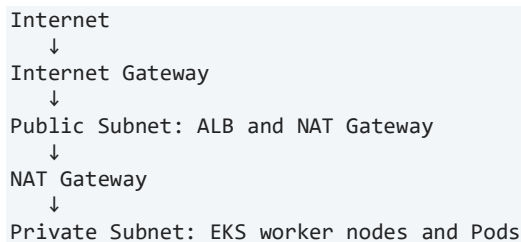
1. AWS Infrastructure Foundation

Completed Architecture

- Completed: VPC
- Completed: Public subnets
- Completed: Private subnets
- Completed: Internet Gateway
- Completed: NAT Gateway

Key point: The networking foundation is ready for Amazon EKS. This is the base architecture used for secure, production-style Kubernetes workloads.

Final Network Architecture



Public-facing resources receive internet traffic in public subnets. Worker nodes and application pods remain in private subnets. The NAT Gateway provides controlled outbound internet access to private workloads.

Interview-ready answer: I implemented a production-grade VPC with public and private subnets across multiple Availability Zones, configured an Internet Gateway for public access, and deployed a NAT Gateway to enable secure outbound internet access for private workloads such as EKS worker nodes.

EKS Prerequisites

Area	Status / Requirement
Networking	VPC, subnets and NAT Gateway completed
IAM roles	Cluster role, node group role and workload roles
Subnet tagging	Required for correct load balancer placement

2. Subnet Tagging for EKS and Load Balancers

Subnet tags guide Kubernetes and AWS Load Balancer Controller when choosing subnets for internet-facing and internal load balancers.

Public Subnet Tag

```
tags = {  
  Name = "${var.env}-public-${count.index}"  
  "kubernetes.io/role/elb" = "1"  
}
```

Meaning: use this subnet for an internet-facing load balancer.

Private Subnet Tag

```
tags = {  
  Name = "${var.env}-private-${count.index}"  
  "kubernetes.io/role/internal-elb" = "1"  
}
```

Meaning: use this subnet for an internal load balancer.

Tag	Purpose
kubernetes.io/role/elb	Internet-facing load balancer
kubernetes.io/role/internal-elb	Private or internal load balancer

Terraform Commands

```
cd ~/production-eks-platform/infra/terraform/envs/dev  
terraform plan  
terraform apply
```

Simple analogy: Subnet tagging is like labeling two rooms as Public and Internal so EKS knows where each type of load balancer belongs.

Interview-ready answer: I tagged public and private subnets appropriately so that EKS and AWS Load Balancer Controller could place internet-facing and internal load balancers in the correct subnets.

3. Kubernetes Core Concepts

Cluster, Control Plane and Worker Nodes

Component	Simple Meaning	Responsibility
Cluster	Complete Kubernetes environment	Contains the control plane and worker nodes
Control Plane	Brain or manager	Maintains cluster state, exposes the API and schedules workloads

Component	Simple Meaning	Responsibility
Worker Node	EC2 worker machine	Runs application pods and containers

Control Plane Components

- API Server: receives Kubernetes API requests
- Scheduler: selects the worker node for a pod
- Controller Manager: reconciles desired and actual state
- etcd: stores Kubernetes cluster state

In Amazon EKS, AWS manages the highly available control plane. The platform team manages workloads and, depending on the operating model, worker node capacity.

Cluster State

Cluster state means the current condition of Kubernetes resources, including nodes, pods, Deployments, Services, namespaces, configuration and rollout status.

Term	Meaning
Desired state	The configuration defined in YAML or Helm
Actual state	What is currently running in the cluster
Reconciliation	Controllers act to make actual state match desired state

```
Desired state: 3 nginx pods
Actual state: 2 running pods
Controller action: create 1 replacement pod
```

Interview-ready answer: Cluster state is the current condition of the Kubernetes cluster, including pods, nodes, Deployments, Services and configuration objects. I define the desired state through YAML or Helm, and Kubernetes controllers continuously reconcile the actual state with that desired state.

4. Workload Hierarchy and Application Flow

```
EKS Cluster
↓
Namespace
↓
Deployment
↓
ReplicaSet
↓
Pod
↓
Container
↓
Application or Microservice
```

Key Responsibilities

Object	Role
Namespace	Logical isolation for applications or environments
Deployment	Defines image, replicas, update strategy and desired state
ReplicaSet	Maintains the required number of pods
Pod	Smallest deployable unit
Container	Runs the actual application process
Service	Provides a stable network endpoint for pods

Microservice to Pod Mapping

Usually, each independent microservice has its own container image and Kubernetes Deployment. One microservice does not necessarily equal one pod because the Deployment may run multiple replicas.

```
4 microservices × 2 replicas = 8 pods
```

```
user-service      → 2 pods
order-service     → 2 pods
payment-service   → 2 pods
notification-service → 2 pods
```

Important correction: Pods do not autoscale by default. Horizontal Pod Autoscaler must be configured. Self-healing also depends on a controller such as a Deployment or ReplicaSet maintaining the desired replica count.

Interview-ready answer: In my EKS implementation, each microservice was packaged as a separate container image and deployed through a Kubernetes Deployment. The Deployment managed one or more pod replicas, while ReplicaSets and controllers maintained the desired replica count.

5. Kubernetes Service and Stable Networking

Pod IP addresses are temporary and can change when pods restart, scale or move to another node. A Kubernetes Service provides a stable virtual IP and DNS name and routes traffic to matching healthy pods.

ClusterIP Service

ClusterIP is accessible only from inside the Kubernetes cluster. It does not expose the application to the public internet.

```
Internal caller
  ↓
Service: support-api, port 80
  ↓
Pod 1: 10.0.11.19:8080
Pod 2: 10.0.12.238:8080
  ↓
Application containers
```

Service Feature	Benefit
Stable virtual IP	Clients do not depend on changing pod IPs
Stable DNS name	Workloads can call the Service by name
Load distribution	Traffic is routed across matching pods
Dynamic endpoints	New and replacement pod IPs are updated automatically

```
kubectl get svc support-api -n dev-app
kubectl get endpointslice -n dev-app
```

Interview-ready answer: I created a ClusterIP Service to provide a stable internal endpoint for the application. The Service exposed port 80 and forwarded requests to healthy support-api pods on container port 8080.

6. ECS and EKS Comparison

Point	Amazon ECS	Amazon EKS
Platform	AWS-native orchestration	Managed Kubernetes

Point	Amazon ECS	Amazon EKS
Main objects	Cluster, Service, Task, Container	Cluster, Deployment, Pod, Service, Node
Learning curve	Lower	Higher
Operational complexity	Lower	Higher
Kubernetes ecosystem	Not applicable	Helm, Argo CD, RBAC, Network Policies and more
Portability	Primarily AWS-oriented	Kubernetes standard across environments
Best fit	Simpler AWS-native container workloads	Platform engineering and Kubernetes-centric workloads

Selection Guidance

- Choose ECS for simpler AWS-native container deployment and lower operational overhead.
- Choose EKS when Kubernetes-native tooling, GitOps, advanced scheduling, platform controls or portability are required.

Interview-ready answer: I understand both ECS and EKS. ECS is a simpler AWS-native container orchestration service, while EKS provides managed Kubernetes. I selected EKS because the platform required Helm, Argo CD, RBAC, Network Policies, Ingress, autoscaling and GitOps-based delivery.

7. Health, Self-Healing and Troubleshooting

Essential Commands

Command	Purpose
kubectl get pods	View pod status and restart count
kubectl describe pod	Check events, scheduling and probe failures
kubectl logs	Read current application logs
kubectl logs --previous	Read logs from the previous crashed container
kubectl get events	Inspect recent cluster events

Command	Purpose
kubectl top pods	View pod CPU and memory usage
kubectl top nodes	View node CPU and memory usage

Readiness and Liveness Probes

Probe	Purpose
Readiness probe	Stops traffic from reaching a pod that is not ready
Liveness probe	Restarts a stuck or unhealthy container

Interview-ready answer: I configured readiness and liveness probes for Kubernetes workloads. The readiness probe ensured that traffic was sent only to ready pods, while the liveness probe allowed Kubernetes to restart unhealthy or stuck containers.

Self-Healing Validation

```
kubectl delete pod <pod-name> -n dev-app
kubectl get pods -n dev-app -w
```

If the Deployment requires two replicas, the ReplicaSet creates a replacement pod after one is deleted or fails.

Interview-ready answer: I validated Kubernetes self-healing by deleting one pod from a Deployment. Since the desired state required two replicas, Kubernetes automatically created a replacement pod and maintained application availability.

8. Resource Management, Metrics and Autoscaling

Node and Pod Resources

A worker node has actual CPU and memory based on its EC2 instance type. Pods do not have separate physical resources; containers consume CPU and memory from the node.

Item	Meaning
Node capacity	Total EC2 vCPU and memory
Container request	Resource amount used by the scheduler for placement

Item	Meaning
Container limit	Maximum resource usage permitted
Pod usage	Combined resource usage of containers in the pod

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "250m"
    memory: "256Mi"
```

Metrics Server

Metrics Server collects node and pod CPU and memory usage through the Kubernetes Metrics API. It enables `kubectl top` and resource-based Horizontal Pod Autoscaler decisions.

```
kubectl top nodes
kubectl top pods -n dev-app
```

Horizontal Pod Autoscaler

HPA automatically adjusts pod replicas based on observed metrics, within configured minimum and maximum boundaries.

```
Minimum pods: 2
Maximum pods: 5
Target CPU utilization: 50%
```

Interview-ready answer: I configured HPA for the application Deployment with minimum and maximum replica boundaries and a target CPU utilization. Metrics Server exposed CPU metrics through the Kubernetes Metrics API.

Pod Scaling vs Node Scaling

Scaling Mechanism	What It Scales	Typical Trigger
HPA	Pod replicas	CPU, memory or custom metrics
Cluster Autoscaler	Managed node group capacity	Unschedulable pods and underutilized nodes
Karpenter	Right-sized EC2 capacity	Workload scheduling requirements

```
CPU load increases
↓
HPA requests more pod replicas
↓
If nodes have capacity, pods are scheduled
↓
If capacity is insufficient, pods remain Pending
↓
Cluster Autoscaler or Karpenter adds worker capacity
```

Interview-ready answer: I handled pod-level and node-level scaling separately. HPA adjusted pod replicas based on metrics, while Cluster Autoscaler or Karpenter adjusted EC2 worker capacity when pods could not be scheduled due to insufficient resources.

9. Rollouts and Rollbacks

Useful Commands

```
kubectl rollout status deployment/support-api -n dev-app
kubectl rollout history deployment/support-api -n dev-app
kubectl rollout undo deployment/support-api -n dev-app
kubectl rollout undo deployment/support-api --to-revision=1 -n dev-app
```

Production Rollback Options

Approach	Use
Manual emergency rollback	Fast recovery through kubectl during an incident
Pipeline rollback	Controlled deployment and automated recovery stages
Argo CD GitOps rollback	Git revert or previous manifest restoration with audit history

Interview-ready answer: For emergency recovery, I used kubectl rollout undo to return a Deployment to a previous revision. In production, I preferred pipeline-driven or Argo CD GitOps rollback so every change was version-controlled, auditable and repeatable.

10. Building and Publishing the support-api Microservice

What the Docker Image Contains

A Docker image packages the complete runnable microservice, not only an API endpoint.

Included in Image	Kept Outside Image
API routes and controllers	Production passwords and AWS access keys

Included in Image	Kept Outside Image
Business logic	Actual RDS or DynamoDB data
Database access code	SQS queues and SNS topics
AWS SDK integration code	EventBridge rules and Step Functions
Dependencies and runtime	Environment-specific infrastructure
Startup command	Runtime secrets and configuration

```

support-api/
  controllers/
    ticket_controller.py
  services/
    ticket_service.py
  repositories/
    ticket_repository.py
  integrations/
    sqs_client.py
    sns_client.py
    eventbridge_client.py
    stepfunction_client.py
  app.py
  requirements.txt
  Dockerfile

```

Real Company Delivery Flow

```

Developer code commit
↓
CI pipeline: build and test
↓
Docker image build
↓
Image scan and push to Amazon ECR
↓
Kubernetes Deployment references image tag
↓
EKS worker node pulls image
↓
Container starts inside a pod

```

Typical Responsibility Split

Developer Responsibilities	Platform / Cloud Responsibilities
API endpoints and business logic	ECR repositories and image policies
Database and integration code	EKS Deployment, Service and Ingress
Unit tests and Dockerfile	ConfigMap, Secret and IRSA

Developer Responsibilities	Platform / Cloud Responsibilities
Application health endpoint	AWS infrastructure and observability

Interview-ready answer: In my project, the Docker image represented the complete runnable microservice. It included API routes, business logic, database access code, AWS SDK integrations, dependencies and runtime configuration. External AWS resources were provisioned separately and accessed at runtime through IAM-secured permissions.

11. Amazon ECR Workflow

Create Repository

```
aws ecr create-repository --repository-name support-api --region ap-south-1 --image-scanning-configuration scanOnPush=true
```

Authenticate, Tag and Push

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION=ap-south-1
ECR_URI=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/support-api

aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com

docker tag support-api:v1 $ECR_URI:v1
docker push $ECR_URI:v1
aws ecr list-images --repository-name support-api --region $REGION
```

Interview-ready answer: I versioned the support-api container image, authenticated Docker with Amazon ECR, pushed the image to a private ECR repository and configured image scanning on push.

12. Deploying support-api to EKS

Deployment Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: support-api
  namespace: dev-app
  labels:
    app: support-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: support-api
  template:
    metadata:
      labels:
        app: support-api
    spec:
      containers:
        - name: support-api
          image: <account-id>.dkr.ecr.ap-south-1.amazonaws.com/support-api:v1
          ports:
            - containerPort: 8080
          env:
            - name: APP_NAME
              value: "support-api"
            - name: APP_VERSION
              value: "v1"
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
```

Apply and Verify

```
kubectl apply -f k8s/support-api/deployment.yaml
kubectl rollout status deployment/support-api -n dev-app
kubectl get pods -n dev-app -l app=support-api -o wide
```

What This Configuration Does

- Runs the support-api:v1 image from Amazon ECR.
- Maintains two pod replicas for availability.
- Runs the application on container port 8080.
- Defines runtime environment variables.

- Sets CPU and memory requests and limits.
- Checks application health through readiness and liveness probes.

Interview-ready answer: I deployed the support-api microservice to EKS using a Kubernetes Deployment manifest. The manifest defined the ECR image, two pod replicas, container port 8080, environment variables, CPU and memory requests and limits, and readiness and liveness probes.

Service Manifest and Internal Test

```
apiVersion: v1
kind: Service
metadata:
  name: support-api
  namespace: dev-app
spec:
  type: ClusterIP
  selector:
    app: support-api
  ports:
    - port: 80
      targetPort: 8080

kubectl apply -f k8s/support-api/service.yaml
kubectl get svc support-api -n dev-app
kubectl get endpointslice -n dev-app

kubectl run curl-support-api --image=curlimages/curl --rm -it --restart=Never -n dev-app --
curl http://support-api/health
```

Expected response: {"app":"support-api","status":"healthy","version":"v1"}

13. AWS Load Balancer Controller, Ingress, OIDC and IRSA

Public Request Flow

```
User
↓
Route 53 DNS
↓
Application Load Balancer
↓
Kubernetes Ingress rule
↓
Kubernetes Service
↓
Healthy Pod IPs
↓
Container on port 8080
↓
support-api application
```

Component Meanings

Component	Meaning
-----------	---------

Component	Meaning
ALB	Public HTTP and HTTPS entry point
AWS Load Balancer Controller	Creates and manages AWS load balancer resources from Kubernetes objects
Ingress	Host and path-based HTTP routing rules
Service	Stable Kubernetes endpoint for pods
Target Group	ALB backend list containing healthy targets
Pod	Running application unit
Container	Actual support-api process

OIDC and IRSA

OIDC provides the trust bridge between an EKS cluster identity and AWS IAM. IRSA associates an IAM role with a Kubernetes Service Account so selected pods can call AWS APIs without storing access keys inside containers.

```

AWS Load Balancer Controller Pod
  ↓ uses
Kubernetes Service Account
  ↓ mapped through IRSA
IAM Role with least-privilege ALB permissions
  ↓ creates and manages
AWS ALB resources

```

Check the Cluster OIDC Issuer

```

aws eks describe-cluster --name dev-eks-cluster --region ap-south-1 --query
"cluster.identity.oidc.issuer" --output text

```

Check Whether the IAM OIDC Provider Exists

```

OIDC_ID=$(aws eks describe-cluster --name dev-eks-cluster --region ap-south-1 --query
"cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)

echo $OIDC_ID
aws iam list-open-id-connect-providers | grep $OIDC_ID || echo "OIDC provider not found"

```

Interview-ready answer: I used IRSA to give AWS permissions to Kubernetes workloads securely. Instead of storing AWS keys in containers or assigning broad permissions to worker nodes, I associated an IAM role with a Kubernetes Service Account through the EKS OIDC provider.

14. Complete Mental Model

Infrastructure layer
VPC → Public/Private Subnets → IGW/NAT → EKS Cluster → Managed Node Group

Kubernetes workload layer
Namespace → Deployment → ReplicaSet → Pods → Containers

Networking layer
User → ALB → Ingress → Service → Healthy Pods → Application

Scaling layer
Metrics Server → HPA scales Pods → Cluster Autoscaler/Karpenter scales Nodes

One-Line Definitions

Term	Remember It As
Cluster	Complete Kubernetes environment
Control Plane	Kubernetes brain, managed by AWS in EKS
Node Group	Configuration and lifecycle group for EC2 worker nodes
Worker Node	EC2 compute machine that runs pods
Deployment	Desired application replicas and rollout configuration
ReplicaSet	Maintains the requested pod count
Pod	Smallest running unit
Container	Actual application process
Service	Stable pod access point
Ingress	HTTP and HTTPS routing rules
ALB	Public entry gate
NAT Gateway	Outbound internet path for private workloads

15. Completed Milestones and Remaining Roadmap

Completed

- Terraform structure and remote state foundation
- VPC with public and private subnets

- Internet Gateway and NAT Gateway
- IAM roles
- EKS cluster and managed node group
- Worker node verification
- Namespace, Deployment, Pods and Service
- Self-healing and rolling update validation
- Readiness and liveness probes
- Metrics Server and HPA concepts
- Amazon ECR repository and image push
- support-api deployment on EKS
- ClusterIP Service and endpoint verification
- AWS Load Balancer Controller, Ingress and public ALB flow

Remaining Production Topics

- Rollback automation strategy
- Helm charts
- Route 53 DNS and ACM TLS certificate
- RBAC and Network Policies
- ConfigMap and secure secrets management
- IRSA for application workloads
- CloudWatch Container Insights and alerting
- Argo CD GitOps
- CI/CD pipeline integration
- FinOps, cleanup and scale-down procedures
- Complete interview architecture story

Interview-ready answer: I implemented the foundation of a production-grade EKS platform using Terraform, created the cluster and managed node group, deployed a real microservice from Amazon ECR, exposed it through Kubernetes networking and an internet-facing ALB, and validated self-healing, scaling, health checks and rollout behavior.

Next actionable step: Continue from the current project state with the next unfinished implementation step. Before creating any chargeable AWS resource, check the expected cost and define the cleanup command.